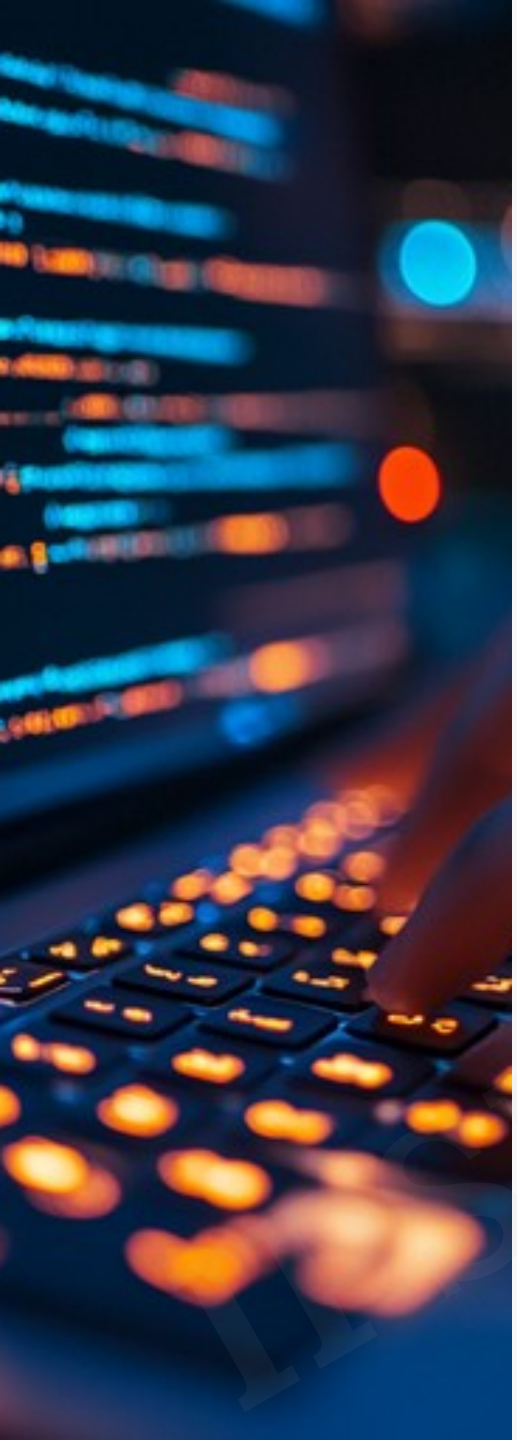


第 4 章、組譯器（Assembler）

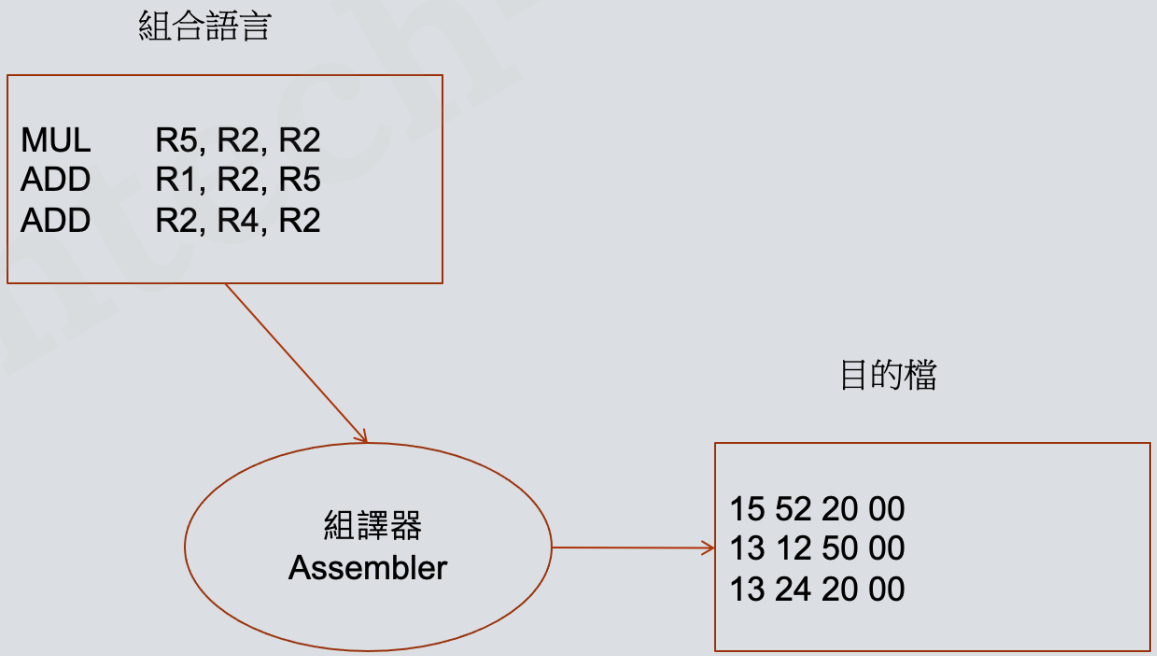
李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

Assembler Introduction

組譯器簡介

- **組譯器**是一個將**組合語言**轉換為**目的檔**的工具，負責將人類可讀的組合語言指令轉換為電腦可執行的**機器碼**。
- 組合語言→ (組譯器) → 目的檔 (或執行檔)



簡單的組合語言程式

▶範例 4.1 簡單的組合語言程式

組合語言程式碼			說明
	LD	R1, B	載入記憶體變數 B 到暫存器 R1 當中
	ST	R1, A	將暫存器 R1 存回記憶體變數 A 當中
	RET		返回
A:	RESW	1	保留一個字組 (Word) 給變數 A
B:	WORD	29	宣告變數 B 為字組, 並設初值為 29

IPSI Lab, Yuntech-CSIE

組譯：將組合語言轉譯成目的碼

▶範例 4.2 為組合語言程式加上目的碼

位址	程式碼			目的碼 (絕對定址版)	目的碼 (相對定址版)
0000		LD	R1, B	00100010	001F000C
0004		ST	R1, A	0110000C	011F0004
0008		RET		2C000000	2C000000
000C	A:	RESW	1	00000000	00000000
0010	B:	WORD	29	0000001D	0000001D

IPSI Lab, Yuntech-CSIE

組譯的原理 (絕對定址)

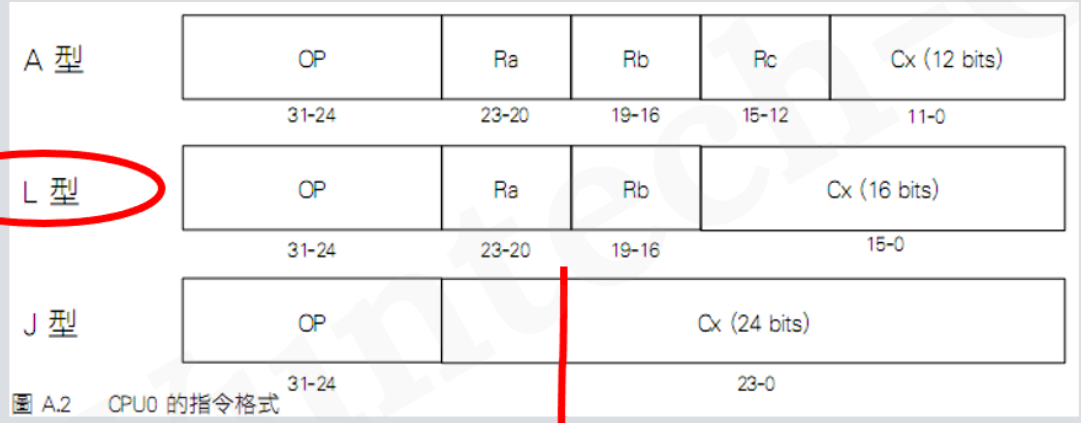


圖 A.2 CPU0 的指令格式

表格 A.1 CPU0 的指令編碼表

類型	格式	指令	OP	說明
載入儲存	L	LD[1]	00	載入 word
	L	ST	01	儲存 word
	L	LDB	02	載入 byte
	L	STB	03	儲存 byte
	A	LDR	04	LD 的暫存器版
	A	STR	05	ST 的暫存器版
	A	LBR	06	LDB 的暫存器版
	A	SBR	07	STB 的暫存器版
	L	LDI	08	立即載入

圖 4.2 將 LD R1, B 指令以絕對定址法編為目的碼

```
    LD      Ra,     [Rb     +       Cx]
=   LD      R1,     R0              B
    00      1       0               0010
```

位址	程式碼		
0000		LD	R1, B
0004		ST	R1, A
0008		RET	
000C	A:	RESW	1
0010	B:	WORD	29

絕對定址的缺點/不合理性

- 因為Cx欄位的大小只有16個 bits，而且採用二補數的格式，最多只能定址-32768~32767的範圍。
  - 最小值 = $-2^{16-1} = -2^{15} = -32768$
  - 最大值 = $2^{16-1} - 1 = 2^{15} - 1 = 32767$
- 絕對位址不可能有負數，於是就只剩 0~32767 的部分可用。
- 變數必須被限制放在 0~32K 的範圍內，這是相當不合理

IPSI Lab, Yuntech-CSIE

組譯的原理 (相對於 PC 定址)

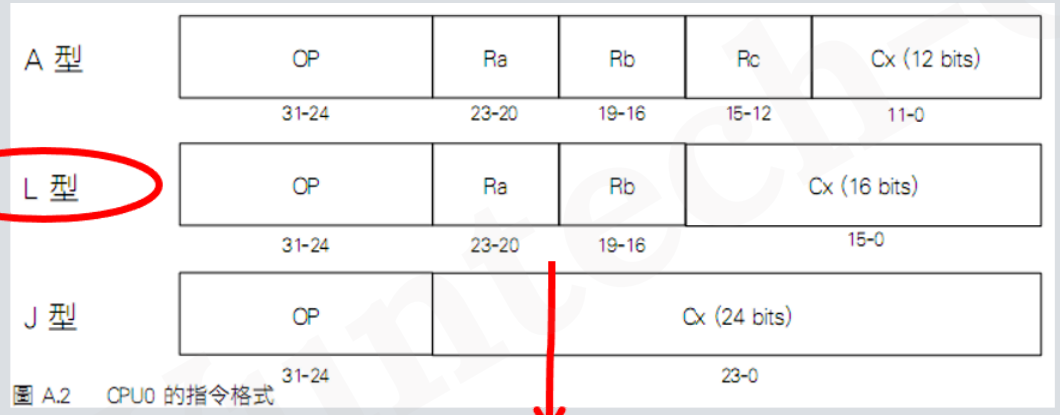


圖 A.2 CPU0 的指令格式

表格 A.1 CPU0 的指令編碼表

類型	格式	指令	OP	說明
載入儲存	L	LD[1]	00	載入 word
	L	ST	01	儲存 word
	L	LDB	02	載入 byte
	L	STB	03	儲存 byte
	A	LDR	04	LD 的暫存器版
	A	STR	05	ST 的暫存器版
	A	LBR	06	LDB 的暫存器版
	A	SBR	07	STB 的暫存器版
	L	LDI	08	立即載入

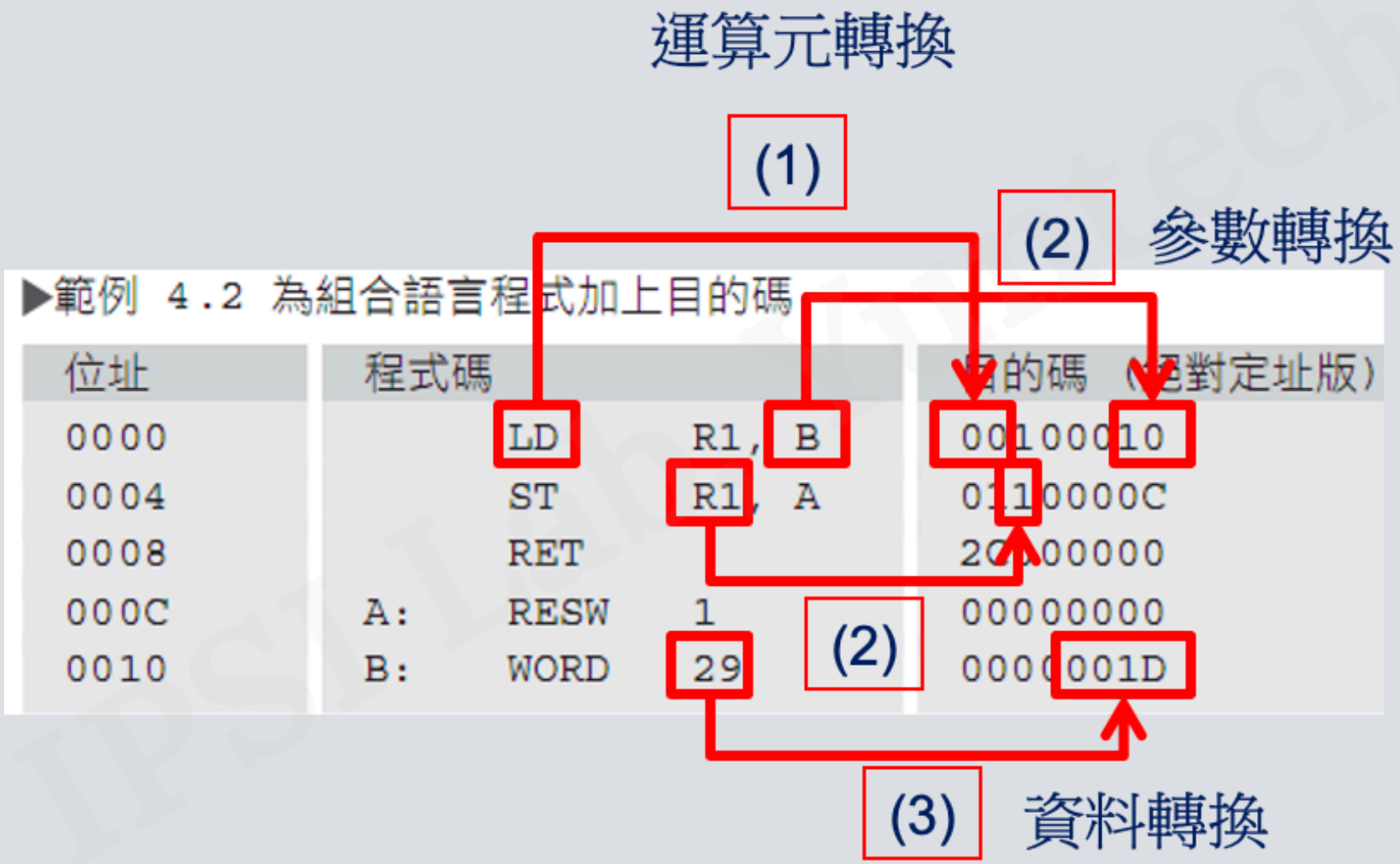
▶圖 4.3 將 LD R1, B 指令以相對於 PC 的方法編為目的碼 R15 = PC

指令編碼						計算 (Cx=B-PC)	
LD	Rd,	[Ra	+	Cx]	Rb	0x0010	(B)
LD	R1,	R15	+	(B-PC)		- 0x0004	(PC)
00	1	F		000C		= 0x000C	(Cx)

位址	程式碼		
0000		LD	R1, B
0004		ST	R1, A
0008		RET	
000C	A:	RESW	1
0010	B:	WORD	28

R15即是PC

二階段的組譯方式


運算元轉換
(1)
(2)
參數轉換
▶範例 4.2 為組合語言程式加上目的碼
位址
程式碼
目的碼 (絕對定址版)
0000 LD R1, B 00100010
0004 ST R1, A 01100000C
0008 RET 20000000
000C A: RESW 1 00000000
0010 B: WORD 29 0000001D
(2)
(3)
資料轉換

組譯步驟

(1) 運算元轉換：

- 將指令名稱轉換為機器語言
- 例如 LD 轉為00, ST 轉為01等

(2) 參數轉換：

- 將暫存器轉為代號，符號轉換成機器位址
- 例如 R1 轉為 1， A 轉為 000C，B 轉為 0010 等。

(3) 資料轉換：

- 將原始程式當中的資料常數轉換為內部的機器碼
- 例如 29 轉換為 001D。

(4) 目的碼產生：

- 根據指令格式，轉換成目的碼，輸出到目的檔中。

- 根據指令格式, 轉換成目的碼, 輸出到目的檔中。

IPSI Lab, Yuntech-CSIE

組譯器二道程序與必要原因

- **組譯器通常包含兩個主要運作階段：**
  - **第一階段**：掃描組合語言程式，**計算每個指令的位址**，並**建立符號表**，將所有的符號（例如變數名稱、標籤）記錄在符號表中，以便在後續的機器碼轉換過程中進行對應。
  - **第二階段**：依照符號表將指令和資料轉換為最終的機器碼。
- **必要原因**
  - 在真正進行組譯之前，必須先計算每一個變數與標記的位址。
  - 範例 4.2，當組譯器對LD R1，B 指令進行編碼動作時，若不知道變數 B 的位址是 0010，就無法順利的編出欄位 Cx 的目的碼。

映像檔

- 映像檔：最簡單的目的碼
- 組合語言→ (組譯器) → 映像檔
- 假如範例 4.2的檔案名稱為 Ex4_1.asm0，範例 4.3 的映像檔就可命名為 Ex4_1.obj0 或 Ex4_1.img。

▶範例 4.2 為組合語言程式加上目的碼

位址	程式碼			目的碼 (絕對定址版)
0000		LD	R1, B	00100010
0004		ST	R1, A	0110000C
0008		RET		2C000000
000C	A:	RESW	1	00000000
0010	B:	WORD	29	0000001D



範例 4.3 範例 4.2 的映像檔 (相對定址版)

```
001F000C 011F0004 2C000000 00000000
0000001D
```

as0 組譯器

▶範例 4.4 使用 as0 組譯器組譯 Ex4_1.asm0

組譯方法與過程	說明
C:\ch12>as0 Ex4_1.asm0 Ex4_1.obj0	組譯 Ex4_1.asm0 輸出到 Ex4_1.obj0
Assembler:asmFile=Ex4_1.asm0 objFile=Ex4_1.obj0	
==========Assemble==========	
LD R1, B	讀入程式並輸出檢視
ST R1, A	
RET	
A: RESW 1	
B: WORD 29	
==========PASS1==========	組譯的第一階段 計算位址 並建立符號表
0000 LD R1, B L 0 (NULL)	
0004 ST R1, A L 1 (NULL)	
0008 RET J 2C (NULL)	
000C A: RESW 1 D F0 (NULL)	
0010 B: WORD 29 D F2 (NULL)	
==========SYMBOL TABLE=========	顯示符號表
000C A: RESW 1 D F0 (NULL)	
0010 B: WORD 29 D F2 (NULL)	
==========PASS2==========	組譯的第二階段 編定目的碼
0000 LD R1, B L 0 001F000C	
0004 ST R1, A L 1 011F0004	
0008 RET J 2C 2C000000	
000C A: RESW 1 D F0 00000000	
0010 B: WORD 29 D F2 0000001D	
==========Save to ObjFile:Ex4_1.obj0==========	將目的碼存入檔案 Ex4_1.obj0
001F000C011F00042C00000000000000000001D	

組譯器的演算法

組譯方法與過程

- 第一階段
  - 計算符號位址

資料結構：符號表

- 第二階段
  - 組譯指令與資料

```
組譯方法與過程
C:\ch12>as0 Ex4_1.asm0 Ex4_1.obj0
Assembler:asmFile=Ex4_1.asm0 objFile=Ex4_1.obj0
==============Assemble============
        LD      R1, B
        ST      R1, A
        RET
A:      RESW    1
B:      WORD    29
================PASS1==============
0000            LD   R1, B          L  0 (NULL)
0004            ST   R1, A          L  1 (NULL)
0008            RET                 J 2C (NULL)
000C A:         RESW 1              D F0 (NULL)
0010 B:         WORD 29             D F2 (NULL)
==============SYMBOL TABLE========
000C A:         RESW 1              D F0 (NULL)
0010 B:         WORD 29             D F2 (NULL)
============PASS2=============
0000            LD   R1, B          L  0 001F000C
0004            ST   R1, A          L  1 011F0004
0008            RET                 J 2C 2C000000
000C A:         RESW 1              D F0 00000000
0010 B:         WORD 29             D F2 0000001D
==========Save to ObjFile:Ex4_1.obj0==========
001F000C011F00042C0000000000000000000001D
```

第一階段 (計算符號位址)

1. **決定每一個指令所佔記憶空間的大小**
   例如
   - 決定 `WORD` 、 `RESW` 等指令所定義的資料長度
   - 決定 `LD` 、 `ST` 等指令所佔空間的大小
2. **計算出程式當中每一行的位址。**
3. **儲存每一個標籤與變數的位址**
   例如
   - 變數 `A` 為 `0x000C`
   - 變數 `B` 為 `0x0010`

組譯方法與過程

```
C:\ch12>as0 Ex4_1.asm0 Ex4_1.obj0
Assembler:asmFile=Ex4_1.asm0 objFile=Ex4_1.obj0
==============Assemble============
        LD      R1, B
        ST      R1, A
        RET
A:      RESW    1
B:      WORD    29
================PASS1==============
0000            LD   R1, B          L  0 (NULL)
0004            ST   R1, A          L  1 (NULL)
0008            RET                 J 2C (NULL)
000C A:         RESW 1              D F0 (NULL)
0010 B:         WORD 29             D F2 (NULL)
==============SYMBOL TABLE========
000C A:         RESW 1              D F0 (NULL)
0010 B:         WORD 29             D F2 (NULL)
=============PASS2=============
0000            LD   R1, B          L  0 001F000C
0004            ST   R1, A          L  1 011F0004
0008            RET                 J 2C 2C000000
000C A:         RESW 1              D F0 00000000
0010 B:         WORD 29             D F2 0000001D
==========Save to ObjFile:Ex4_1.obj0==========
001F000C011F00042C0000000000000000000001D
```

第二階段 (組譯指令與資料)

(1) 轉換指令OP欄位為機器碼

- EX.
  - LD 轉換為 00
  - ST 轉換為 01

(2) 轉換指令參數為機器碼

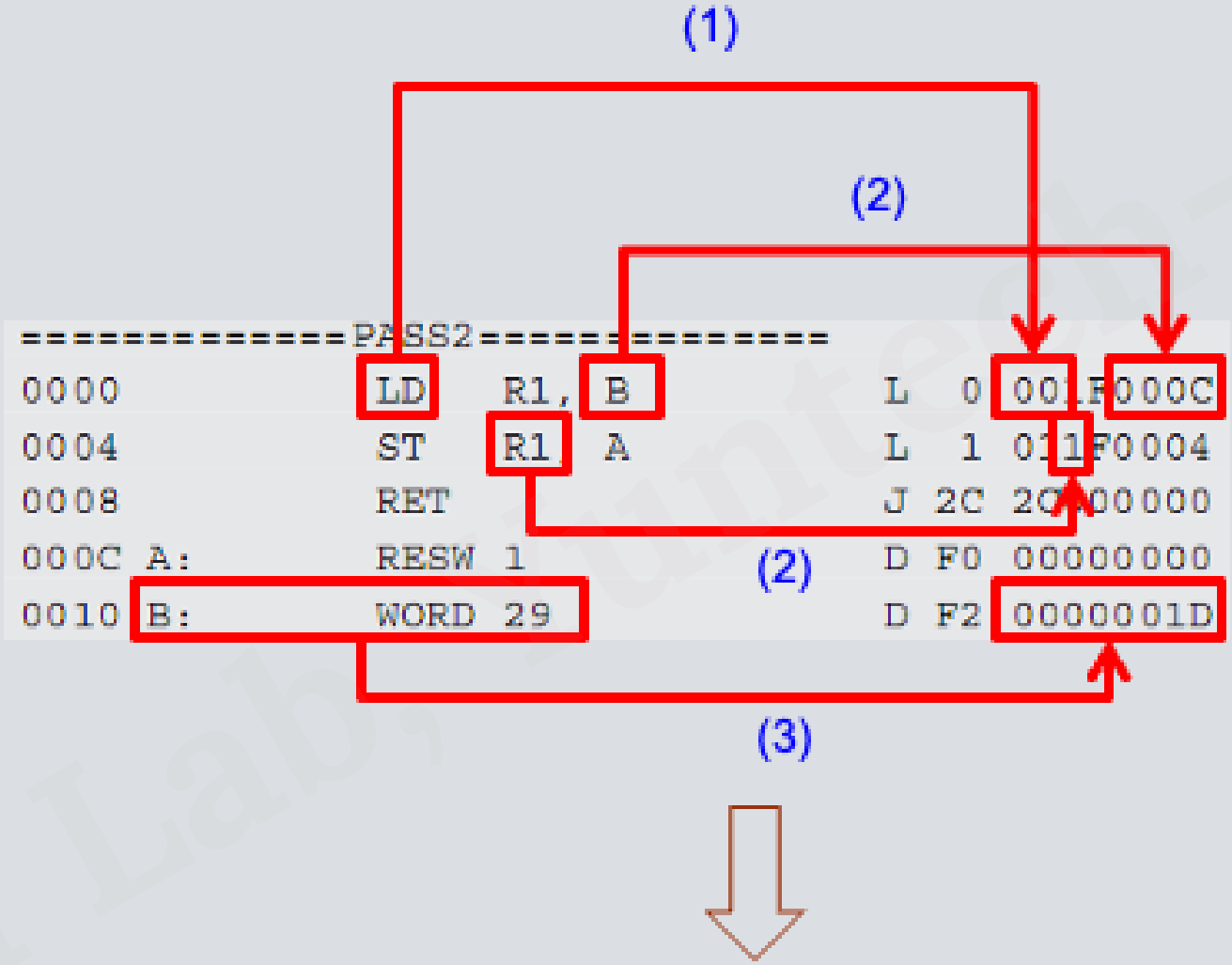
- EX.
  - R1轉換為 1
  - B 轉換為 000C

(3) 轉換資料定義指令為位元值

- EX.
  - WORD B 29 轉為 0x0000001D
  - 29 (decimal) = 1D (hex)

(4) 產生目的碼並輸出到目的檔當中

IPSI Lab, Yuntech-CSIE




```
==========Save to ObjFile:Ex4_1.obj0==========
001F000C011F00042C0000000000000000001D
```

將目的碼存入檔案 Ex4_1.obj0

組譯器的資料結構

- 運算碼表 (Op Table)
  - 儲存指令與代碼的配對表
  - 通常使用雜湊表格
- 符號表 (Symbol Table)
  - 儲存符號與位址的配對表
  - 通常使用雜湊表格

範例 4.5 資料結構 1 - 指令表

LD	00
ST	01
LDB	02

範例 4.6 資料結構 2 - 符號表

A	0000000C
B	00000010

IPSI Lab, Yuntech-CSIE

第一階段(PASS1)演算法- 計算位址

```
Algorithm AssemblerPass1                    // 組譯器的第一階段演算法
Input AssemblyFile                          // 輸入：組合語言檔案
Output SymbolTable                          // 輸出：符號表

    SymbolTable = new Table();              // 建立空的符號表
    file = open(AssemblyFile)               // 開啟組合語言檔案
    while not file.end                      // 當檔案還沒結束時，繼續讀取
        line = readLine(file)               // 讀取下一行
        if line is comment                  // 如果該行為註解
            continue                        // 則忽略此行，繼續下一輪迴圈
        label = label(line)                 // 取得該行的標籤
        if label is not null                // 如果該行有標籤
            symbolRecord = symbolTable.search(label) // 於符號表中尋找該標籤
            if symbolRecord is not null     // 如果找到該標籤
                report error                // 則報告錯誤，標記重複定義
            else
                symbolTable.add(label, address) // 將（標籤、位址）放入符號表中
        op = operator(line)                 // 取得該行中的指令部分（操作碼）
        opRecord = opTable.search(op);      // 於指令表中尋找該指令
        if opRecord is not null             // 如果找到該指令
            address += 4;                   // 則將位址加上 4
        else if op is 'BYTE'                // 如果指令為 BYTE
            address += 1 * length(parameters) // 則將位址加上 1 乘以參數個數
        else if op is 'WORD'                // 如果指令為 WORD
            address += 4 * length(parameters) // 則將位址加上 4 乘以參數個數
        else if op is 'RESB'                // 如果指令為 RESB
            address += length(parameter 1) // 則將位址加上參數 1 的長度
        else if op is 'RESW'                // 如果指令為 RESW
            address += 4 * length(parameter 1) // 則將位址加上 4 乘以參數 1 的長度
        else
            report error                    // 代表該指令拼寫有誤，顯示錯誤訊息
    end while                               // 結束迴圈
End Algorithm                               // 結束演算法
```

目的

- 逐行處理組合語言檔案，建立符號表並計算每個標記（如標籤和變數名稱）的位址。
- 過程中，根據不同的指令調整位址，並檢查重複定義或無效指令的錯誤。

輸入與輸出

輸入:

- 組合語言檔案（AssemblyFile）

輸出:

- 符號表（SymbolTable）
    - 符號表包含組合語言程式碼中所有標記（如標籤和變數名稱）及其對應的位址資訊。

IPSI Lab, Yuntech-CSIE

演算法步驟

1. **初始化與開啟檔案**:
   - 首先，建立一個空的符號表來儲存程式碼中的標籤資訊。
   - 開啟要處理的組合語言檔案，準備逐行讀取其內容。
2. **逐行解析程式碼**:
   - 使用迴圈逐行讀取組合語言檔案中的內容，直到檔案讀取完畢。
   - 每讀取一行，判斷該行是否為註解行：
     - 如果是註解行，則略過該行並繼續讀取下一行。

IPSI Lab, Yuntech-CSIE

3. **處理標籤（Label）**：
    - 對於每一行程式碼，檢查是否包含標籤。
    - 如果該行有標籤，則：
        - 在符號表中查詢該標籤是否已經存在：
            - 如果標籤已存在，代表該標籤重複定義，需報告錯誤。
            - 如果標籤不存在，將該標籤與其對應的位址新增到符號表中。

IPSI Lab, Yuntech-CSIE

4. **處理指令**:
   - 取得該行中的操作碼（也就是指令部分），並在操作碼表中查詢是否有符合的指令。
   - 根據指令的類型，進行以下位址調整：
     - 如果指令有效（標準指令），加上 4 個位元組。
     - 如果指令為 `BYTE` ，則將位址加上 1 乘以參數的數量。
     - 如果指令為 `WORD` ，則將位址加上 4 乘以參數的數量。
     - 如果指令為 `RESB` ，則根據參數值增加位址。
     - 如果指令為 `RESW` ，則將位址加上 4 乘以參數值。
     - 如果指令無法識別或拼寫錯誤，需報告錯誤並停止處理。
5. **結束**:
   - 當檔案中的所有行皆已處理完畢，演算法結束。

第二階段 (PASS2) – 主程式(指令轉為機器碼)

```
Algorithm AssemblerPass2                    // 組譯器的第二階段演算法
Input AssemblyFile, SymbolTable             // 輸入：組合語言檔案，符號表
Output ObjFile                              // 輸出：目的檔

    file = open(AssemblyFile)               // 開啟組合語言檔案作為輸入
    while not file.end                      // 當檔案尚未結束時，繼續讀檔
        line = readLine(file)               // 讀取下一行
        (op, parameter) = parse(line)       // 取得指令碼與參數
        opRecord = opTable.search(op)       // 在指令表中查找該指令
        if opRecord is not null             // 如果找到該指令
            objCode = translateInstruction(parameter, address, opRecord) // 將指令轉換為目的碼
            address += length(objCode) // 計算下一個指令的位址
        else if op is 'WORD' or 'BYTE' // 如果指令是 WORD 或 BYTE
            objCode = translateData(line) // 將資料轉換為目的碼
            address += length(objCode) // 計算下一個指令的位址
        else if op is 'RESB'                // 如果指令為 RESB
            address += parameter[1]     // 將位址加上參數 1
        else if op is 'RESW'                // 如果指令為 RESW
            address += 4 * parameter[1] // 將位址加上 4 倍的參數 1
        end if
        output ObjCode to ObjFile       // 將目的碼寫入目的檔案
    end while                           // 結束迴圈
End Algorithm                           // 結束演算法
```

目的

- 逐行讀取組合語言程式碼，**根據指令和符號表將其翻譯為目的碼**，並依指令類型更新位址，確保轉換準確。
- 將所有轉換後的目的碼依序**寫入目的檔案**，完成組譯並生成可執行的檔案。

輸入與輸出

- **輸入**
  - 組合語言檔案 (`AssemblyFile`)
  - 符號表 (`SymbolTable`)
    - 符號表包含組合語言程式碼中所有標記（如標籤和變數名稱）及其對應的位址資訊。
    - 此表格在第二階段用來解析和確保每個指令中標記的正確性。

- **輸出**
  - **目的檔案 (`ObjFile`)**:
    - 演算法生成的目的是將組合語言檔案中的指令和資料轉換為機器可執行的目的碼，並將這些目的碼寫入到目的檔案中。
    - 這個檔案包含已編譯的二進位碼或位元碼，能夠被硬體或模擬器執行。

IPSI Lab, Yuntech-CSIE

演算法步驟

1. **開啟檔案**
2. **逐行讀取**:
   - 依序讀取檔案中的每一行程式碼，直到檔案的所有行都被讀取完畢。
3. **解析指令與參數**:
   - 將讀取到的每一行程式碼解析為指令和對應的參數，這有助於辨識該行的動作和資料。
4. **查找指令**:
   - 在指令表中查找該指令是否存在。如果找到指令，則進行後續的目的碼轉換過程。

IPSI Lab, Yuntech-CSIE

5. **指令翻譯**:
    - 若指令存在，根據符號表及指令的參數將該指令轉換為對應的目的碼。此步驟確保指令被正確編譯。
6. **位址更新**:
    - 依據轉換後的目的碼長度，更新當前位址，確保下一個指令位址的正確性。

```
objCode = translateInstruction(parameter, address, opRecord) // 將指令轉換為目的碼
address += length(objCode) // 計算下一個指令的位址
```

IPSI Lab, Yuntech-CSIE

7. **特殊指令處理**:
   - 如果指令不是常規指令（例如 `WORD`、`BYTE`、`RESB`、`RESW`），根據其特性進行特殊處理：
     - **`WORD` 和 `BYTE` 會被視為資料存取指令，需將其資料轉換為目的碼並更新位址。**

       ```
       objCode = translateData(line) // 將資料轉換為目的碼
       address += length(objCode) // 計算下一個指令的位址
       ```

     - **`RESB` 會保留特定數量的位元組空間，根據參數更新位址。**
     - **`RESW` 會保留特定數量的字組空間（每字組為 4 位元組），依據參數計算新位址。**

8. **輸出目的碼**:
    - 將每一行轉換後的目的碼逐一寫入目的檔案中，以形成完整的輸出結果。
9. **迴圈結束**:
    - 持續上述步驟，直到檔案中的每一行都處理完成。
10. **結束演算法**:
    - 當所有行都處理完畢後，結束整個演算法流程，最終生成一個完整的目的檔案。

IPSI Lab, Yuntech-CSIE

第二階段 (PASS2) – 指令轉換函數

```
Algorithm TranslateInstruction           // 將指令轉換為目的碼
Input parameter, pc, opRecord            // 輸入：參數、程式計數器、指令記錄
Output objCode                           // 輸出：目的碼

    if (opRecord.type is L)              // 如果是 L 型指令
        Ra = parameter[1]                // 設定 Ra 為參數
        if (parameter[3] is Constant)    // 如果參數 3 是常數
            Rb = 0
            Cx = toInteger(parameter[2]); // 設定 Cx 為該常數
        else if (parameter[3] is Variable) // 如果參數 3 是變數
            Cx = address(Variable) - pc   // 設定 Cx 為位移 (標記位址 - PC)
            Rb = parameter[2]            // 設定 Rb 為參數
        end if
        objCode = opRecord.opCode
                + id(Ra) + id(Rb) + hex(Cx); // 設定目的碼

    else if (op.type is A)               // 如果是 A 型指令
        Ra = parameter[1]                // 取得 Ra
        Rb = parameter[2]                // 取得 Rb
        if (parameter[3] is Register)    // 如果參數 3 是暫存器
            Rc = parameter[3]            // 取得 Rc
        else if (parameter[3] is Constant) // 如果參數 3 是常數
            Cx = toInteger(parameter[3]); // 設定 Cx 為該常數
        end if
        objCode = opRecord.opCode
                + id(Ra) + id(Rb) + id(Rc) + hex(Cx); // 設定目的碼

    else if (op.type is J)               // 如果是 J 型指令
        Cx = parameter[1]                // 取得 Cx
        objCode = opRecord.opCode + hex(Cx) // 設定目的碼
    end if

    return objCode                       // 傳回目的碼
End Algorithm
```

目的:

- 依據指令的類型（`L`、`A`、`J`）及其參數的特性（常數、變數或暫存器），來組合出相應的目的碼。
- 確保每個指令都能被正確地轉換成機器可執行的目的碼。

輸入和輸出

輸入:

- `parameter`：指令的參數，用來判斷和計算目的碼的細節。
- `pc`（程式計數器）：表示目前指令的位址，用來計算變數的位址相對位移。
- `opRecord`：指令記錄，包含指令的類型（例如 `L` 型、`A` 型、`J` 型）以及對應的操作碼。

輸出:

- `objCode`：生成的目的碼，即將指令及其參數翻譯後的機器碼，用來在硬體上執行。

演算法步驟

1. **判斷指令類型**:
   - 根據輸入的指令記錄 (`opRecord`) 來判斷指令的類型，有三種主要類型：
     - **L 型指令**：處理包含暫存器和常數或變數的指令。
     - **A 型指令**：處理包含三個參數的指令，通常是運算指令。
     - **J 型指令**：跳躍指令，僅包含位址資訊。

IPSI Lab, Yuntech-CSIE

2. **處理 L 型指令**:
   - 取出指令的第一個參數設為 `Ra`，作為主要的暫存器。
   - 如果第三個參數是常數，則直接使用該常數來設定位址偏移量 (`Cx`) 並將 `Rb` 設為 0。
   - 如果第三個參數是變數，計算變數位址相對於程式計數器 (`pc`) 的位移，並將該結果作為偏移量 (`Cx`)，同時設置 `Rb` 為第二個參數的值。
   - 根據這些參數組合目的碼。

IPSI Lab, Yuntech-CSIE

3. **處理 A 型指令**:
   - 取出第一個和第二個參數，分別設為 `Ra` 和 `Rb`。
   - 如果第三個參數是暫存器，將其設為 `Rc`；如果是常數，則將其轉換並設為 `Cx`。
   - 將操作碼和這些參數組合為目的碼。
4. **處理 J 型指令**:
   - 直接取出第一個參數設為 `Cx`，代表跳躍的位址或目標。
   - 將操作碼和這個參數組合為目的碼。
5. **生成並回傳目的碼**:
   - 根據不同的指令類型和其參數，演算法生成相應的目的碼，並最終回傳這個目的碼。

範例

C語言陣列加總

```c
int a[] = {3, 7, 4};  // 陣列 a
int *aptr = a;        // 指標 aptr 指向陣列 a 的第一個元素
int sum = 0;          // 初始化 sum 為 0

for (int i = 3; i > 0; i--) {  // 從 i=3 開始，每次迴圈 i 減 1，直到 i = 0
    sum += *aptr;              // 將 aptr 所指的值加到 sum
    aptr += 1;                 // 將指標移到下一個元素（因為 int 型別大小為 4 bytes）
}

return sum;  // 返回 sum 的值
```

IPSI Lab, Yuntech-CSIE

組合語言陣列加總

```
        LDI     R1, 0               ; R1=0
        LD      R2, aptr            ; R2=aptr
        LDI     R3, 3               ; R3=3
        LDI     R4, 4               ; R4=4
        LDI     R9, 1               ; R9=1
FOR:
        LD      R5, [R2]            ; R5=*aptr
        ADD     R1,R1,R5            ; R1+=*aptr
        ADD     R2, R2, R4          ; R2+=4
        SUB     R3, R3, R9          ; R3--;
        CMP R3, R0                  ; if (R3!=0)
        JNE     FOR                         ;   goto FOR;
        ST      R1, sum             ; sum=R1
        LD      R8, sum             ; R8=sum
        RET
a:      WORD    3, 7, 4 ; int a[]={3,7,4}
aptr:   WORD    a       ; int *aptr = &a
sum:    WORD    0       ; int sum = 0
```

IPSI Lab, Yuntech-CSIE

Assemble結果

```
==============Assemble=============
        LDI     R1, 0               ; R1=0
        LD      R2, aptr            ; R2=aptr
        LDI     R3, 3               ; R3=3
        LDI     R4, 4               ; R4=4
        LDI     R9, 1               ; R9=1
FOR:
        LD      R5, [R2]            ; R5=*aptr
        ADD     R1,R1,R5            ; R1+=*aptr
        ADD     R2, R2, R4          ; R2+=4
        SUB     R3, R3, R9          ; R3--;
        CMP R3, R0                  ; if (R3!=0)
        JNE     FOR                             ;   goto FOR;
        ST      R1, sum             ; sum=R1
        LD      R8, sum             ; R8=sum
        RET
a:      WORD    3, 7, 4 ; int a[]={3,7,4}
aptr:   WORD    a       ; int *aptr = &a
sum:    WORD    0       ; int sum = 0
```

IPSI Lab, Yuntech-CSIE

```
=================PASS1===============
0000           LDI  R1, 0           L  8 (NULL)
0004           LD   R2, APTR        L  0 (NULL)
0008           LDI  R3, 3           L  8 (NULL)
000C           LDI  R4, 4           L  8 (NULL)
0010           LDI  R9, 1           L  8 (NULL)
0014 FOR:                             FF (NULL)
0014           LD   R5, [R2]        L  0 (NULL)
0018           ADD  R1,R1,R5        A 13 (NULL)
001C           ADD  R2, R2, R4      A 13 (NULL)
0020           SUB  R3, R3, R9      A 14 (NULL)
0024           CMP  R3, R0          A 10 (NULL)
0028           JNE  FOR             J 21 (NULL)
002C           ST   R1, SUM         L  1 (NULL)
0030           LD   R8, SUM         L  0 (NULL)
0034           RET                  J 2C (NULL)
0038 A:        WORD 3, 7, 4         D F2 (NULL)
0044 APTR:     WORD A               D F2 (NULL)
0048 SUM:      WORD 0               D F2 (NULL)
===============SYMBOL TABLE=========
0044 APTR:     WORD A               D F2 (NULL)
0048 SUM:      WORD 0               D F2 (NULL)
0038 A:        WORD 3, 7, 4         D F2 (NULL)
0014 FOR:                             FF (NULL)
```

IPSI Lab, Yuntech-CSIE

```
============PASS2==============
0000            LDI   R1, 0             L  8 08100000
0004            LD    R2, APTR          L  0 002F003C
0008            LDI   R3, 3             L  8 08300003
000C            LDI   R4, 4             L  8 08400004
0010            LDI   R9, 1             L  8 08900001
0014 FOR:                                 FF
0014            LD    R5, [R2]          L  0 00520000
0018            ADD   R1,R1,R5          A 13 13115000
001C            ADD   R2, R2, R4        A 13 13224000
0020            SUB   R3, R3, R9        A 14 14339000
0024            CMP   R3, R0            A 10 10309000
0028            JNE   FOR               J 21 21FFFFE8
002C            ST    R1, SUM           L  1 011F0018
0030            LD    R8, SUM           L  0 008F0014
0034            RET                     J 2C 2C000000
0038 A:         WORD 3, 7, 4            D F2 000000030000000700000004
0044 APTR:      WORD A                  D F2 00000038
0048 SUM:       WORD 0                  D F2 00000000
```

```
==========Save to ObjFile:Arraysum.obj0==========
08100000002F003C0830000308400004089000010052000013115000132240001433900010309000021FFFFE8011F0018008F00142C0000000000000300000007
0000000400000038000000000
請按任意鍵繼續 . . .
```

IPSI Lab, Yuntech-CSIE

VM執行結果

```
==VM0:run arraysum.obj0 on CPU0===
PC=00000004 IR=08100000 SW=00000000 R[01]=0X00000000=0
PC=00000008 IR=002F003C SW=00000000 R[02]=0X00000038=56
PC=0000000C IR=08300003 SW=00000000 R[03]=0X00000003=3
PC=00000010 IR=08400004 SW=00000000 R[04]=0X00000004=4
PC=00000014 IR=08900001 SW=00000000 R[09]=0X00000001=1
PC=00000018 IR=00520000 SW=00000000 R[05]=0X00000003=3
PC=0000001C IR=13115000 SW=00000000 R[01]=0X00000003=3
PC=00000020 IR=13224000 SW=00000000 R[02]=0X0000003C=60
PC=00000024 IR=14339000 SW=00000000 R[03]=0X00000002=2
PC=00000028 IR=10309000 SW=00000000 R[12]=0X00000000=0
PC=00000014 IR=21FFFFE8 SW=00000000 R[15]=0X00000014=20
PC=00000018 IR=00520000 SW=00000000 R[05]=0X00000007=7
PC=0000001C IR=13115000 SW=00000000 R[01]=0X0000000A=10
PC=00000020 IR=13224000 SW=00000000 R[02]=0X00000040=64
PC=00000024 IR=14339000 SW=00000000 R[03]=0X00000001=1
PC=00000028 IR=10309000 SW=00000000 R[12]=0X00000000=0
PC=00000014 IR=21FFFFE8 SW=00000000 R[15]=0X00000014=20
PC=00000018 IR=00520000 SW=00000000 R[05]=0X00000004=4
PC=0000001C IR=13115000 SW=00000000 R[01]=0X0000000E=14
PC=00000020 IR=13224000 SW=00000000 R[02]=0X00000044=68
PC=00000024 IR=14339000 SW=00000000 R[03]=0X00000000=0
PC=00000028 IR=10309000 SW=40000000 R[12]=0X40000000=1073741824
PC=0000002C IR=21FFFFE8 SW=40000000 R[15]=0X0000002C=44
PC=00000030 IR=011F0018 SW=40000000 R[01]=0X0000000E=14
PC=00000034 IR=008F0014 SW=40000000 R[08]=0X0000000E=14
PC=00000038 IR=2C000000 SW=40000000 R[00]=0X00000000=0
```

```
===CPU0 dump registers===
IR =0x2c000000=738197504
R[00]=0x00000000=0
R[01]=0x0000000e=14
R[02]=0x00000044=68
R[03]=0x00000000=0
R[04]=0x00000004=4
R[05]=0x00000004=4
R[06]=0x00000000=0
R[07]=0x00000000=0
R[08]=0x0000000e=14
R[09]=0x00000001=1
R[10]=0x00000000=0
R[11]=0x00000000=0
R[12]=0x40000000=1073741824
R[13]=0x00000000=0
R[14]=0xffffffff=-1
R[15]=0x00000038=56
請按任意鍵繼續 . . .
```

IPSI Lab, Yuntech-CSIE